

Asymmetric Data-Quality Sensitivity in Predictive Models and LLM Agents

Amrutha Satishkumar
San Jose, United States
amruthashetty299@gmail.com

Krithika Padmanabha Suvarna
San Jose, United States
krithikap019@gmail.com

Abstract

Data quality is conventionally treated as a property of the data alone: a table is validated once, and a single verdict declares it clean or not. We argue that quality is instead a system-level property that depends on the downstream consumer. Existing work in data validation, large language model (LLM) evaluation, and robustness benchmarking holds the consumer fixed, and even synthetic-corruption benchmarks measure the impact on a single model. In contrast, we route one curated table through two heterogeneous consumers, a gradient-boosted tree (XGBoost) and an LLM analyst agent (gpt-4o-mini), and observe how each degrades under a shared library of corruption families applied at multiple severities.

We study a tabular-only prediction task and a second task that additionally exposes a free-text channel to the agent. The two consumers disagree, often sharply, on which corruptions matter most: schema-identity drift is catastrophic for the tree on the tabular task yet leaves the agent essentially unaffected, while on the text-bearing task the asymmetry reverses, because renamed column identifiers flow directly into the agent’s prompt.

Across the full sweep, the per-cell AUROC drops the two consumers produce agree only weakly: a single consumer’s verdict explains roughly 2% of the variance in the other’s, and the same corruption family flips which consumer it hurts most when the task is changed. A single-consumer quality verdict therefore cannot be transferred across heterogeneous consumers, with direct implications for the design of data validation and LLM-evaluation pipelines.

***Index Terms.** data quality, large language models, robustness, schema drift, paired evaluation, consumer-dependent quality, gradient-boosted trees.*

I. INTRODUCTION

A single curated table rarely serves a single purpose. The same table of e-commerce orders may feed a gradient-boosted tree trained on its tabular features and, separately, an LLM analyst agent that reads the same rows serialised as JSON, in-

cluding a free-text customer review. Now suppose an upstream extract-transform-load job silently renames a fraction of the feature columns to opaque identifiers. The tabular pipeline, whose preprocessing pipeline learnt its column vocabulary at fit time, sees an all-zero feature vector for the renamed columns and its AUROC collapses; the LLM agent, which receives column names verbatim in its prompt, can still read the renamed columns as strings and is largely unaffected. A column-level validator inspecting the table in isolation would either flag this change for both consumers or for neither, because it has no notion of which consumer reads which channel. The verdict that matters, “Is this clean for you?,” differs between the two, and no single answer is correct.

This is the blind spot we address. Conventional tooling judges quality on the data alone, treating it as an intrinsic property answerable without reference to who will use it. We argue instead that quality is a system-level property of the consuming system: which corruptions count as damage, and how much, are determined by the consumer, not the column. A consumer-independent verdict therefore misranks corruptions whenever one source feeds heterogeneous consumers, and the misranking is not a corner case: in our sweep it is the typical outcome.

This gap persists across otherwise unrelated lines of work, all of which hold the consumer fixed. Data-validation systems judge the table in isolation: DEEQU frames quality as declarative, software-style data checks [1], and TensorFlow Data Validation detects anomalies in the data fed to a pipeline [2], neither conditioning the verdict on a particular consumer. The cost of neglecting data work is well documented: data cascades from upstream issues were reported by 92% of practitioners in one large study [3]. LLM evaluation benchmarks [4], [5] measure capability on clean inputs rather than robustness to upstream drift, and robustness benchmarks [6]–[8] vary the input distribution for one fixed model per task. Closest to our setting, JENGA [9] perturbs tabular data with synthetic corruptions, including missing values, outliers, and accidental rescaling, and measures the impact on a model’s predictions, but it studies the robustness of a single model. None of these compare two heterogeneous consumers reading the same corrupted data, which is precisely the comparison that exposes consumer-dependent quality.

We make three contributions.

- 1) A *paired-clean* evaluation protocol that scores each consumer on the same rows in clean and corrupted form, so every effect isolates the corruption rather than sampling noise, with five seeds per cell giving paired bootstrap confidence intervals and a signed-rank test.
- 2) A *two-task corruption sweep* across a tabular consumer (XGBoost) and an LLM agent (gpt-4o-mini), on tasks with and without a text channel, spanning six corruption families and three severities, with paired uncertainty quantified in every one of the 150 cells.
- 3) An *analysis of consumer asymmetry*: some corruptions collapse the tree while the agent is invariant, others drive the agent below its clean baseline while the tree is unaffected, and some are absorbed by both. 76% of the 150 cells are substantively asymmetric (one consumer hurt at least $2\times$ more than the other), and the cross-cell Spearman rank correlation is small ($\rho = 0.15$, $\rho^2 = 2.3\%$), so a quality verdict from one consumer is not a usable proxy for the other.

We are deliberate about the scope of these claims. The empirical contribution of this paper is the existence and direction of consumer asymmetry on one representative tabular benchmark (the public Brazilian e-commerce dataset), evaluated with two specific consumers (a fixed XGBoost configuration and a single LLM agent based on gpt-4o-mini). Replication across additional domains (for example, electronic health records or financial transactions) and across alternative consumer families (other tree learners, stronger reasoning LLMs, retrieval-augmented agents) is left to follow-up work. What we claim to demonstrate here is that consumer-dependent quality is observable on a non-trivial benchmark, not that the specific per-cell rankings will transfer unchanged to every domain or every consumer.

II. RELATED WORK

A. Data validation and data-centric AI

A long line of work treats data quality as a property of the table. Polyzotis et al. surveyed the data lifecycle in production ML [10]; Schelter et al. proposed DEEQU, a declarative library of data unit tests [1]; and Breck et al. described TensorFlow Data Validation, which detects anomalies between batches in a serving pipeline [2]. The data-centric AI program [11], [12] re-emphasised that systematic data improvements often outweigh model-architecture changes. Industry commentary echoes the same prescription [13], and a large-scale practitioner study documented the resulting “data cascades” [3]. What unifies this body of work is that the verdict it produces, whether a numeric anomaly score or a Boolean check, is not conditioned on the downstream consumer. Our work asks instead whether such consumer-free verdicts are transferable when one curated source feeds heterogeneous consumers.

B. LLM evaluation

LLM benchmarks measure the model’s competence on clean inputs. HELM provides a multi-axis evaluation across many tasks but does not vary the input distribution adversarially [4]; MT-Bench studies chatbot quality with an LLM-as-judge protocol [5]. A growing body of work asks whether LLMs can act as tabular consumers directly: TabLLM uses few-shot prompting for tabular classification [14], and Borisov et al. explored LLMs as tabular generators [15]. Our agent follows the TabLLM tradition of feeding a serialised row plus a training-set profile to a chat-completion model, but we use the agent as a downstream *consumer* of corrupted data rather than as the object of evaluation.

C. Robustness and synthetic corruption

The robustness literature varies the input distribution to measure how a fixed model degrades. ImageNet-C introduced a fixed library of common image corruptions [7]; Taori et al. showed that synthetic robustness does not always predict natural-shift robustness [8]; WILDS catalogued in-the-wild distribution shifts for a range of modalities [6]. On tabular data, JENGA introduced a parameterised library of corruption operators (missing values, scaling errors, categorical swaps) and measured the resulting drop in a single model’s predictions [9]. Our sweep adopts the same paired-clean idea but evaluates two heterogeneous consumers on the same corrupted rows, which is what allows us to ask whether a corruption ranking is transferable between consumers at all.

III. METHODOLOGY

A. Dataset

We build on the public Brazilian e-commerce dataset released by Olist via Kaggle [16], comprising approximately 99,441 orders placed between September 2016 and September 2018. The release is structured as nine normalised tables covering orders, line items, payments, reviews, customers, sellers, and products, together with a Portuguese-to-English product-category map.

We join these tables on their order, customer, and product keys and aggregate all line-level fields to a single row per order, yielding one curated table that serves as the shared input to both consumers. Its columns span the order timeline (purchase, approval, carrier-handoff, and delivery timestamps, with derived deltas such as days-to-estimated-delivery and approval lag); aggregated economics (order and freight totals, freight ratio, item count, payment value, and instalments); geography (customer and seller state and a same-state indicator); product attributes (category, weight, and volume); and, where available, the customer review score and free-text review comment. The review fields are read only on the task that admits a text channel; the tabular consumer never sees free text.

TABLE I
THE TWO PREDICTION TASKS.

	T1: late delivery	T2: negative review
Label	days-to-deliver > days-to-estimate	review score ≤ 2
Inputs	tabular only	tabular + review text
Test rows	18,603	17,584
Positive rate	5.5%	11.8%
Purpose	isolates pure tabular asymmetry (no text channel)	tests text-channel vulnerability (strong text signal)

All experiments use a single frozen 60/15/25 split by order purchase timestamp, not a random split, to prevent later orders from leaking into training. The earliest 60% train the tabular model and supply the agent’s feature profiles, the next 15% fix hyperparameters and the decision threshold, and the final 25% form the held-out test partition. Because the two tasks define their targets on different subsets of orders (orders without a delivery date are excluded from T1; orders without a review are excluded from T2), this partition holds 18,603 orders for the late-delivery task and 17,584 orders for the negative-review task. Sweep cells subsample from this partition rather than scoring it in full, drawing a class-balanced sample per cell.

B. Prediction Tasks

We define two binary classification tasks on the held-out test partition, chosen to bracket the design space along a single structural axis: T1 has no text channel at prediction time, while T2 exposes the original Portuguese review comment alongside the tabular features. Both consumers are evaluated on each task. Table I summarises the labels, input channels, and class balance.

T1 is tabular-only and favours the tabular consumer; T2 adds a text channel that the agent can exploit but the tree cannot. This contrast is what lets a single corruption fall on a channel one consumer depends on while the other never reads it.

C. Downstream Predictors

The two consumers are designed to be structurally different in exactly one respect: their access to the row’s free-text content. Their implementation, frozen state, and inference settings are summarised in Table II; the prose below describes the methodologically relevant points that the table does not capture.

1) *Consumer-ML (XGBoost pipeline)*: The tree never reads the free-text review comment, even on T2 where the text is present in the row. Numeric and categorical features pass through a preprocessing pipeline, fit once on clean training data and held fixed for every cell, that median-imputes numerics and most-frequent-imputes plus one-hot-encodes categoricals; categories unseen at fit time map to the all-zero vector. The training-time medians and category

TABLE II
SIDE-BY-SIDE SPECIFICATION OF THE TWO DOWNSTREAM CONSUMERS.

	Tree (ML)	Agent (LLM)
Implementation	XGBoost [17]	gpt-4o-mini via Azure OpenAI
Reads text?	no	yes (rendered into prompt)
Reads table?	yes (numeric/categorical)	yes (JSON-serialised row)
Frozen state	preprocessing pipeline (medians, one-hot vocab)	training-set profile (per-feature stats)
Hyperparameters	300 trees, depth 6, $\eta = 0.05$, subsample 0.9, $\lambda = 1.0$	temperature 0, seed 13, 200-token cap
Refit on corrupted data?	no	no (no fine-tuning, no retrieval, no tools)
Output	class probability + thresholded label	JSON {prediction, confidence, rationale}
Score for AUROC	probability	confidence field
Failure mode	none (always emits a score)	parse failure (0% in 120,000 calls)

vocabulary play a role analogous to the agent’s static profile: both summarise the clean training distribution, and neither is recomputed on corrupted data. Because the pipeline coerces and imputes deterministically, the tree always returns a well-formed score, so every corrupted-row evaluation contributes to the metrics in Section IV.

2) *Consumer-Agent (LLM with profiled prompt)*: The agent receives, alongside each candidate row, a fixed training-time profile of the input features. The profile is intended to provide distributional context analogous to what the tree absorbs implicitly during fitting. For numeric features it includes sample size, missingness rate, mean, standard deviation, the 10th, 50th, and 90th percentiles, minimum, maximum, and Pearson correlation with the label; for categorical features it includes cardinality, missingness rate, the three modal levels with counts, and the conditional label prevalence at the modal level. The profile is computed once on the clean training split and held fixed throughout the sweep.

Each prediction comes from a single chat-completion request whose user message contains five labelled blocks in a fixed order: the task definition and label semantics, one-line feature descriptions, the training-set profile, the (potentially corrupted) row serialised as JSON, and an output-format reminder. Free-text review fields are truncated to 800 characters to bound prompt length on T2, and missing values are rendered as null. Responses are parsed by a strict JSON validator; the binary prediction field is used for F_1 and the continuous confidence field is used as the agent’s AUROC score. The agent is not fine-tuned, performs no retrieval beyond the static profile, invokes no external tools, and does not use self-consistency sampling or post hoc confidence calibration. These choices intentionally isolate the effect of upstream corruption on a fixed LLM-based consumer and are revisited in Section IV-F.

TABLE III
CORRUPTION FAMILIES USED IN THE MAIN SWEEP.

Family	Mechanism	Tasks
missing_injection	Replaces feature values with NaN, modelling partial joins, optional-field gaps, and source-system dropouts.	T1, T2
currency_unit_drift	Multiplies monetary fields (e.g. total_price, total_freight) by 100, modelling a cents-versus-reais unit bug.	T1, T2
schema_drift	Renames feature columns to opaque identifiers col_seed_idx, modelling upstream schema changes not propagated downstream.	T1, T2
type_flip	Coerces numeric values to their string representation, e.g. 199.9 → "199.9", modelling schema-on-read type drift.	T1, T2
mojibake_roundtrip	Mis-decodes UTF-8 review and category text through Latin-1, e.g. não → nÃo, modelling an encoding mistake on the text channel.	T2
category_taxonomy_drift	Replaces Portuguese product-category values with plausible variants, e.g. bebês → bebê, modelling taxonomy and normalisation drift.	T2

D. Corruption Library

We evaluate downstream robustness using a fixed library of data-quality corruptions. Each corruption family is implemented as a deterministic transformation $f_c(D, s, k) \rightarrow D'$, where D is the feature table, $s \in \{0.10, 0.20, 0.30\}$ is the severity, and k is the random seed. All corruptions preserve row count and label values; target columns, split identifiers, and order identifiers are never modified. With the exception of schema_drift, which renames a fraction of feature columns, the transformations also preserve the input schema. Corruptions are applied after the evaluation rows are materialised and before they are presented to either downstream consumer, so neither the ML model nor the LLM agent observes the clean version of a corrupted row during scoring.

For row-level corruptions, severity is the fraction of rows affected. For schema_drift, severity is the fraction of feature columns renamed. This range was chosen to capture moderate but nontrivial data-quality faults: large enough to be measurable, but not so large that both consumers collapse into uninformative noise.

Table III summarises the six corruption families used in the main sweep, and Table IV lists their task coverage. T1 is evaluated on the four tabular families because it has no review-text channel; T2 is evaluated on all six families because it admits both tabular and text inputs.

The corruption library is designed to support paired evaluation rather than adversarial worst-case testing. Each corrupted condition is compared against a clean reference on the same

TABLE IV
TASK-FAMILY COVERAGE AND TOTAL CELLS IN THE MAIN SWEEP.

Task	Families	Severities	Seeds	Cells
T1 (late-delivery)	4	3	5	60
T2 (negative-review)	6	3	5	90
Total				150

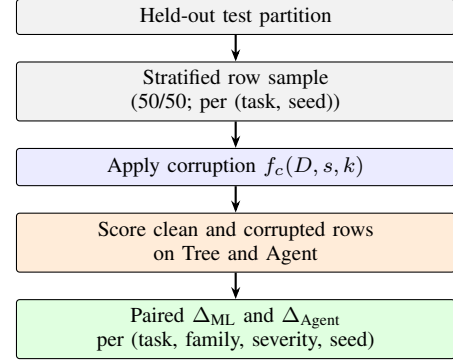


Fig. 1. Per-cell evaluation pipeline: sample rows → apply one corruption $f_c(D, s, k) \rightarrow$ score clean and corrupted on both consumers → paired AUROC delta.

sampled rows, as described in Section III-E. This isolates the effect of the corruption family from sampling variation and allows the two downstream consumers to be compared under identical input perturbations.

Each cell is generated deterministically from a fixed random seed. The implementation includes unit tests for determinism, label preservation, severity-zero identity, and row-count preservation. Family-specific invariants are also checked, including a missing-value increase under missing_injection, monetary-field scaling under currency_unit_drift, and exact rename counts under schema_drift.

E. Experimentation

We evaluate both consumers under a single controlled protocol designed to isolate the effect of a corruption from the noise of sampling. The full sweep comprises 150 cells, where a cell is one combination of task, corruption family, severity, and seed, and every cell is scored on both consumers in paired clean and corrupted forms. Figure 1 summarises the per-cell pipeline; we describe the protocol and the seed-level statistics in turn.

1) *Paired-clean protocol*: A cell does not score the entire test partition. Instead, it draws a stratified evaluation sample from the held-out test split, 1,000 rows for T2 and 500 rows for T1, balanced 50/50 on the target. Subsampling is necessary because scoring the full partition through the LLM agent on every cell is cost-prohibitive, and the 50/50 stratification stabilises per-cell AUROC given the low positive rates of the raw data (approximately 11.8% on T2 and 5.5% on T1), which would otherwise produce noisy estimates.

Within a cell, the same sampled rows are scored twice, once in clean form and once after the corruption is applied, by both consumers. Because the two conditions share identical rows, the per-cell effect is a paired difference,

$$\Delta\text{AUROC} = \text{AUROC}_{\text{corrupt}} - \text{AUROC}_{\text{clean}},$$

which cancels the sampling variance that would otherwise dominate a sample of this size. This paired construction is what makes a single cell’s effect attributable to the corruption rather than to which rows happened to be drawn, and it is the protocol that the bootstrap and signed-rank statistics below operate on. Clean baselines are cached per (task, seed) so that the clean score is computed once and reused across the corruption families that share a seed. Corruption is applied to the input rows only; both consumers’ fitted state (the tree together with its preprocessing pipeline, and the agent together with its training-set profile) is computed once on clean training data and is never recomputed on corrupted data, and the temporal split is frozen throughout.

2) *Statistics:* For each (task, family, severity) condition we obtain five seed-level paired deltas, and the following statistics summarise them. Our primary uncertainty measure is a paired bootstrap [18] confidence interval: we resample the paired deltas with $B = 2,000$ resamples to produce a 95% interval on the mean ΔAUROC . We treat this interval, rather than a p -value, as the primary evidence that an effect is non-trivial.

We additionally report the Wilcoxon signed-rank test [19] on the seed-level deltas and Cohen’s d_z [20], the paired standardised mean difference, defined as the mean delta divided by the sample standard deviation of the deltas. We are explicit about a constraint of the five-seed design: with $n = 5$, the smallest achievable two-sided signed-rank p -value is 0.0625, so the test cannot reach significance at the 0.05 level regardless of how consistent the effect is. We therefore interpret $p = 0.0625$ as the strongest available signal, namely that all five seeds moved in the same direction, and we rely on the bootstrap interval and on d_z , which on our strongest effects reaches roughly four standard units, as the primary magnitude evidence. Five seeds was a deliberate budget choice; additional seeds would tighten the intervals and are left to future work.

Two further statistics operationalise the asymmetry that is the focus of this work. The *asymmetry ratio*, $|\Delta_{\text{Agent}}| / |\Delta_{\text{ML}}|$ with a small floor to handle invariant consumers, quantifies how differently a single corruption strikes the two consumers; we treat a ratio of at least 2.0 as a substantively asymmetric effect. Because the ratio is computed within each cell on paired data, seed-to-seed variation cancels. Finally, to test whether the two consumers agree on the ordering of corruptions overall, we compute the Spearman rank correlation between Δ_{ML} and Δ_{Agent} across all 150 cells. This cross-cell correlation, reported in Section IV-D, is the headline test of consumer-dependent quality: it asks not whether a given corruption hurts, but whether the two consumers rank corruptions the same way at all.

TABLE V
MEAN CLEAN-CONDITION AUROC BY TASK AND CONSUMER, AVERAGED ACROSS ALL SWEEP CELLS. THE AGENT IS AT CHANCE ON T1 BECAUSE THAT TASK HAS NO TEXT CHANNEL; THE TREE IS HELD BELOW THE AGENT ON T2 BECAUSE IT CANNOT ENCODE FREE TEXT.

Task	Consumer-ML	Consumer-Agent
T1 (tabular only)	0.702	0.494
T2 (tabular + text)	0.613	0.826

IV. RESULTS

We report results in five parts. Section IV-A fixes the clean baselines so subsequent deltas can be read in context. Section IV-B examines the tabular-only task and shows that schema-identity drift is by far the dominant tabular failure mode. Section IV-C examines the text-bearing task, on which the asymmetry reverses. Section IV-D reports the cross-cell rank correlation, the headline test of consumer-dependent quality. Section IV-E reports the per-cell asymmetry ratio, and Section IV-F states the threats to validity.

A. Clean baselines

Table V reports the mean clean-condition AUROC for each (task, consumer) combination, averaged over the five seeds and (where applicable) the corruption families that share a seed. The two tasks expose a strong consumer asymmetry even before any corruption is applied. On T1, where no text channel is available, the tree reaches an AUROC of 0.702, whereas the agent is at chance (0.494): a generic LLM classifier with a careful prompt and a statistical profile cannot beat random on a tabular-only task drawn from this domain. On T2, the asymmetry reverses, but for a different reason: the agent reads the free-text review comment, where most of the predictive signal lives, and reaches 0.826 AUROC; the tree, which never sees the text, is held to 0.613. We will use these baselines, not zero, as the reference for interpreting subsequent ΔAUROC values.

B. T1: schema-identity drift dominates the tabular failure surface

Table VII reports the mean paired delta on T1 for each (family, severity) cell, and Table VI pulls out the two non-trivial effects on the tree with their bootstrap intervals, paired effect sizes, and worst-seed AUROCs. *schema_drift* is the dominant family: at high severity its bootstrap interval excludes zero and its worst seeds drag the tree’s AUROC below 0.42. *missing_injection* is the second-strongest effect, with a clean dose-response across severities and the largest $|d_z|$ observed anywhere in the sweep. The agent’s mean deltas on the same cells are statistically indistinguishable from zero, but the appropriate reading is not “the agent absorbs the corruption”: it is that the agent is already at chance on T1, so its score has nowhere left to fall. *currency_unit_drift* is absorbed by the tree because median imputation on the unscaled training

TABLE VI
HEADLINE EFFECTS ON T1 (CONSUMER-ML). PAIRED BOOTSTRAP
 $B = 2000$, 95%. CLEAN BASELINE AUROC = 0.702.

Family (sev.)	mean Δ	95% CI	d_z	worst
schema_drift (hi)	-0.153	[-0.27, -0.04]	-1.65	0.365
missing_inj. (med)	-0.046	[-0.06, -0.03]	-4.04	0.642
missing_inj. (hi)	-0.072	[-0.09, -0.05]	-3.51	0.612

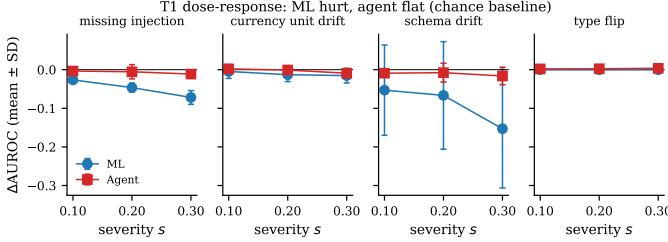


Fig. 2. T1 dose-response: paired Δ AUROC vs. severity (mean $\pm$ SD over five seeds). Tree drops on missing_injection and schema_drift; agent stays at chance throughout.

distribution attenuates the $\times 100$ shift in monetary fields, and type_flip is absorbed by both consumers because the tabular preprocessing pipeline re-coerces strings to numerics and the agent is type-tolerant in its JSON serialisation.

Figure 2 plots the dose-response across the four T1 families for both consumers. The two non-trivial effects on the tree (missing_injection and schema_drift) are visible as descending curves with separable error bars; currency_unit_drift and type_flip sit on the zero line. The agent’s curves are flat within their error bars across all four families.

The per-seed picture for the dominant family is informative: at high severity, three of the five seeds push the tree’s Δ AUROC below -0.17 (worst -0.311), while every agent score remains within ± 0.05 . The variance across seeds at all three severities reflects which subset of feature columns the random seed selects for renaming: when one of the high-importance timestamp-derived columns is hit, the tree collapses regardless of severity (seed 17 already drops to -0.26 at the low setting); when the renaming falls on lower-importance columns, the impact is small. This within-condition variance is what motivates the paired-clean protocol: the bootstrap CI in Table VII reflects exactly this seed dispersion.

C. T2: schema and missingness now hurt the agent more

Table IX reports the same cells on T2, and Table VIII pulls out the cells where the consumer most hurt has flipped relative to T1. Two reversals are worth noting. First, on schema_drift, the agent now takes the bigger hit: $|\Delta_{\text{Agent}}|$ exceeds $|\Delta_{\text{ML}}|$ by roughly $3\times$ at medium severity. The mechanism is that column identifiers flow into the agent’s prompt: when they are renamed to opaque col_seed_idx strings, the TRAINING-SET PROFILE block (whose column names were not renamed because the profile is frozen at training time) and the ROW block stop

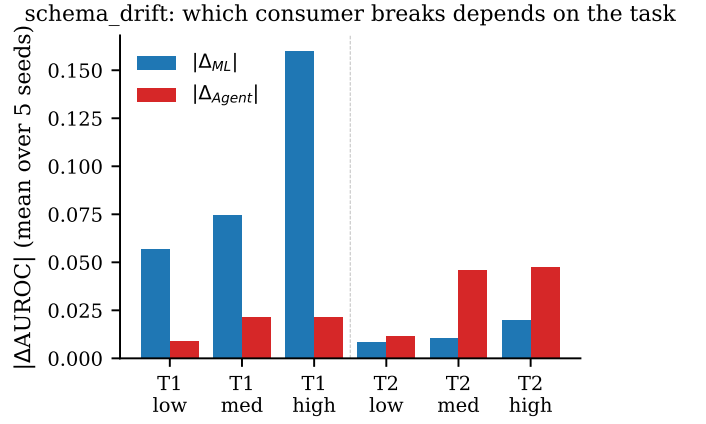


Fig. 3. schema_drift flips its target across tasks: tree dominates on T1, agent dominates on T2 (mean $|\Delta$ AUROC| over five seeds).

sharing keys, and the agent loses the alignment that lets it look up a row’s value in the profile. The tree, by contrast, sees only its frozen column vocabulary; renamed columns drop out of its feature space and are silently treated as absent, which costs only a small amount of AUROC because most of T2’s signal lives in the review text, which the tree never reads anyway.

Second, on missing_injection, the agent is hurt more than the tree at every severity, unlike on T1. We attribute this to the agent’s reliance on the row’s textual content as well as its tabular content: replacing a value with null in the JSON-serialised row removes information the agent was using, whereas the tree’s median imputation is only mildly disruptive on a task where its baseline signal is weak to start with.

The two text-channel families are interesting precisely because they are small. mojbake_roundtrip costs the agent at most 0.004 AUROC; category_taxonomy_drift costs at most 0.004. Modern LLMs are evidently robust to limited Latin-1 mis-decoding of Portuguese text and to plausible spelling variants of category labels: both corruption types leave the underlying lexical signal mostly intact, and the agent recovers it. We had hypothesised, before running the sweep, that the agent would degrade most on text-channel corruptions. The data do not support that hypothesis: the dominant agent-hurting families on T2 are the structural ones (schema and missingness), not the textual ones.

Figure 3 makes the schema-drift reversal explicit by plotting $|\Delta_{\text{ML}}|$ and $|\Delta_{\text{Agent}}|$ side by side for each (task, severity) cell of schema_drift. On T1 the left bar (tree) dominates; on T2 the right bar (agent) dominates. The same corruption is asymmetrically damaging in both directions, depending only on which task it is paired with.

D. Cross-cell rank correlation

We complement the per-family results above with a global agreement statistic. Table X reports the Spearman correlation between the two consumers’ per-cell AUROC drops across all

TABLE VII

MEAN Δ AUROC ON T1 (LATE-DELIVERY), AVERAGED OVER FIVE SEEDS. NEGATIVE NUMBERS INDICATE A DROP FROM CLEAN. “↓” MARKS CELLS WHOSE PAIRED BOOTSTRAP 95% INTERVAL EXCLUDES ZERO; “~” MARKS CELLS IN WHICH THE INTERVAL COVERS ZERO. TREE AUROC HAS MORE ROOM TO FALL ON T1 BECAUSE THE AGENT IS ALREADY AT CHANCE.

Family	Consumer-ML			Consumer-Agent		
	low	med	high	low	med	high
missing_injection	−0.027↓	−0.046↓	−0.072↓	−0.003~	−0.005~	−0.011↓
currency_unit_drift	−0.005~	−0.013~	−0.015↓	+0.002~	−0.001~	−0.009↓
schema_drift	−0.053~	−0.067~	−0.153↓	−0.009↓	−0.008~	−0.016~
type_flip	0.000~	0.000~	0.000~	+0.002~	+0.002~	+0.004~

TABLE VIII

HEADLINE REVERSALS ON T2. BOTH CONSUMERS ARE EVALUATED ON THE SAME ROWS; $r = |\Delta_{\text{AGENT}}|/|\Delta_{\text{ML}}|$.

Family (sev.)	ML Δ	Agent Δ	r
schema_drift (med)	−0.010	−0.041	4.1
schema_drift (high)	−0.017	−0.042	2.5
missing_injection (med)	−0.017	−0.021	1.2
missing_injection (high)	−0.024	−0.033	1.4

150 cells. The substantive finding is the effect size, not the test outcome: $\rho^2 = 2.3\%$, so the tree’s per-cell drop explains only about 2% of the variance in the agent’s per-cell drop, leaving 97.7% unexplained. The two-sided p -value of 0.064 does not cross $\alpha = 0.05$, but the conclusion does not rely on rejecting the null: even if $\rho = 0.15$ were declared statistically significant with more cells, it would remain a weak rank agreement, and a consumer-blind verdict would still misrank corruptions for the second consumer most of the time. The headline evidence for consumer-dependent quality is the per-family sign-reversal of Section IV-C and the per-cell asymmetry ratio of Section IV-E; the rank correlation is the global summary that those per-cell effects do not cancel into agreement when aggregated. Figure 4 plots the per-cell $(\Delta_{\text{ML}}, \Delta_{\text{Agent}})$ pairs: the cells cluster into two off-diagonal arms (T1 cells extending leftward on the Δ_{ML} axis, T2 cells extending downward on the Δ_{Agent} axis), with very few cells on the agreement diagonal.

E. Per-cell asymmetry ratio

The cross-cell correlation summarises the global picture. To pinpoint which cells contribute the asymmetry, we compute the per-cell asymmetry ratio $r = |\Delta_{\text{Agent}}|/|\Delta_{\text{ML}}|$ with a 10^{-4} floor on the denominator and bucket the 150 cells (Table XI). Only 24% of cells fall in the symmetric band; the remaining 76% are asymmetric, and the asymmetric cells are not distributed uniformly across the sweep. Tree-asymmetric cells ($r \leq 0.5$) are concentrated on T1 (missing_injection and schema_drift); agent-asymmetric cells ($r \geq 2.0$) are concentrated on T2 (schema_drift, missing_injection, and the small text-channel corruptions where the tree by construction has a zero delta).

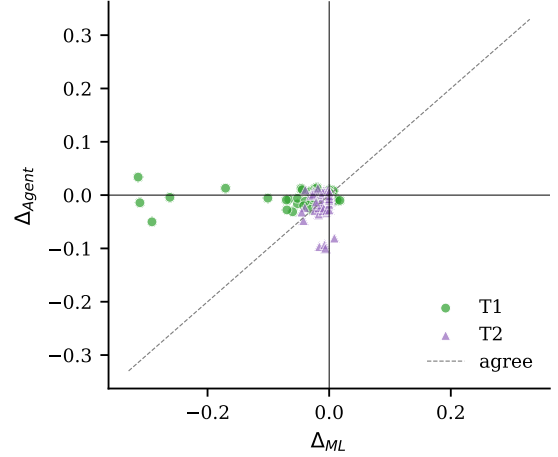
Per-cell paired deltas, all 150 cells ($\rho^2 = 2.3\%$)

Fig. 4. Per-cell paired deltas, all 150 cells: T1 cells extend along the Δ_{ML} axis, T2 cells along the Δ_{Agent} axis; the agreement diagonal is largely empty (Spearman $\rho = 0.15$, $\rho^2 = 2.3\%$: one consumer’s verdict explains roughly 2% of the variance in the other’s).

F. Threats to validity

i. *Single dataset, single domain.* All experiments are on Brazilian e-commerce data [16]. The specific magnitudes of every effect we report should be read as within-domain estimates. The qualitative claim, namely that two heterogeneous consumers do not agree on a corruption ordering, would need replication on a second domain (for example, electronic health records or financial transactions) to be advanced as a general law of data quality; we leave this to follow-up work.

ii. *Two specific consumers.* Our tabular consumer is one fixed XGBoost configuration, and our agent is gpt-4o-mini at temperature zero. A different tree (e.g. LightGBM with deeper trees) might be more or less sensitive to schema rename through its preprocessing pipeline, and a different LLM (a stronger reasoning model, or one with explicit retrieval) might recover from schema_drift by treating column identifiers as soft hints. The asymmetry pattern in our data is between two specific consumers; the existence of asymmetry is the claim we believe will generalise, the specific direction within a single (task, family) cell is the claim that will not.

iii. *Statistical power.* Five seeds is a small n . The Wilcoxon

TABLE IX

MEAN Δ AUROC ON T2 (NEGATIVE-REVIEW). THE ASYMMETRY IS INVERTED RELATIVE TO T1: SCHEMA_DRIFT NOW HURTS THE AGENT MORE THAN THE TREE, AND MISSING_INJECTION HURTS THE AGENT MORE AT EVERY SEVERITY.

Family	Consumer-ML			Consumer-Agent		
	low	med	high	low	med	high
missing_injection	-0.008↓	-0.017↓	-0.024↓	-0.012↓	-0.021↓	-0.033↓
currency_unit_drift	-0.010↓	-0.013↓	-0.017↓	-0.005~	-0.007~	-0.001~
schema_drift	-0.008~	-0.010↓	-0.017↓	-0.007~	-0.041↓	-0.042↓
type_flip	0.000~	0.000~	0.000~	-0.006↓	-0.002~	-0.003~
mojibake_roundtrip	0.000~	0.000~	0.000~	-0.004↓	-0.003~	+0.001~
category_taxonomy_drift	-0.002↓	-0.002~	-0.001~	-0.004↓	-0.004~	-0.003↓

TABLE X

CROSS-CELL AGREEMENT BETWEEN THE TWO CONSUMERS (ALL 150 CELLS).

Statistic	Value
Cells (T1 + T2)	60 + 90 = 150
Spearman $\rho(\Delta_{ML}, \Delta_{Agent})$	0.152
Two-sided p value	0.064
Variance explained (ρ^2)	2.3%
Significant at $\alpha = 0.05$?	no

TABLE XI

DISTRIBUTION OF THE PER-CELL ASYMMETRY RATIO
 $r = |\Delta_{AGENT}|/|\Delta_{ML}|$ ACROSS ALL 150 CELLS.

Bucket	Cells	Share
Tree hurt $\geq 2\times$ more ($r \leq 0.5$)	43	28.7%
Approximately symmetric ($0.5 < r < 2.0$)	36	24.0%
Agent hurt $\geq 2\times$ more ($r \geq 2.0$)	71	47.3%
Total	150	100%

signed-rank test bottoms out at $p = 0.0625$, so we cannot achieve $\alpha = 0.05$ significance from the test alone, and we instead lean on paired bootstrap intervals and Cohen’s d_z . We made this trade explicit in Section III-E; adding seeds is the natural follow-up.

iv. *Sub-sampled evaluation.* Each cell evaluates 500 T1 rows or 1,000 T2 rows rather than the full test partition, because scoring the full partition through the LLM agent on every cell is cost-prohibitive. The paired construction cancels the resulting sampling variance for the Δ AUROC of a *single cell*; it cannot cancel the variance of the absolute clean baseline, which is why we read the deltas relative to Table V rather than to a held-out absolute reference.

v. *No retraining or recalibration.* Both consumers’ fitted state is frozen at training time. We do not retrain, fine-tune, or calibrate either consumer on corrupted data. This isolates the upstream-data effect, which is the goal, but it also means the numbers we report should not be read as predicting how either consumer would behave in a system that re-fits on observed drift.

vi. *Threats to internal validity.* The two consumers see the same sampled rows and the same corruption operator within

a cell. The clean baseline is cached per (task, seed) so that all corruption families for a given seed are compared against the same reference. The agent’s fitted “state” is its training-set profile, which is computed once on the clean training split and never recomputed. With these protocols in place, we attribute the deltas in this section to the corruption family rather than to confounders.

V. CONCLUSION

We posed a single question: when two heterogeneous consumers read the same corrupted table, do they agree on which corruption is worst? Across 150 paired evaluation cells, on a tabular-only and a text-bearing classification task drawn from the same curated source, the answer is *no*. 76% of the cells are substantively asymmetric, with one consumer hurt at least twice as much as the other, and schema-identity drift flips which consumer it strikes hardest depending on whether the task exposes a text channel: it collapses the gradient-boosted tree on the tabular-only task while leaving the LLM agent untouched, and on the text-bearing task it hurts the agent roughly three times more than the tree. The cross-cell rank agreement between the two consumers is correspondingly weak ($\rho = 0.15$, $\rho^2 = 2.3\%$). `type_flip` is absorbed by both consumers; `mojibake` and `category-taxonomy` drifts cost the agent at most 0.005 AUROC each and are absorbed by the tree by construction.

The takeaway for system design is concrete. Data-validation tooling that emits a single pass/fail verdict on a table is, at best, a verdict for one consumer; at worst, it is a verdict for none. LLM-evaluation pipelines that score a fixed model on clean inputs miss a class of failures that appear only when the same data is also routed through a tabular consumer on a related task. The natural follow-up is a quality-checking layer that takes the consumer as an explicit input, and that produces, for each candidate corruption, a per-consumer impact estimate rather than a single table-level verdict. Building that layer, and validating it on a domain beyond Brazilian e-commerce, is the direction we intend to pursue.

CODE AND DATA AVAILABILITY

All code, configuration files, and the frozen temporal split used in this paper are available at <https://github.com/amruthasatishkumar/AsymmetricDataQualitySensitivity>. The study uses the public Olist Brazilian E-Commerce dataset (CC BY-NC-SA 4.0), which the repository downloads and processes from raw CSVs into the single 99,441-row silver table that both consumers receive. Every numeric claim in Section IV can be regenerated from logs/sweep.parquet via the `verify_numbers.py` script shipped with the manuscript sources. Corruption variants are deterministic functions of (family, severity, seed); LLM calls use temperature=0 with full request/response logging.

GENERATIVE AI USE DISCLOSURE

During manuscript preparation, the authors used generative AI tools (ChatGPT and GitHub Copilot) solely to assist with minor language editing, including grammar, spelling, and sentence-level clarity. No generative AI tools were used to produce, design, or interpret the experiments, the data analysis, or the scientific claims of this paper. The authors take full responsibility for the content of the manuscript.

REFERENCES

- [1] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, “Automating large-scale data quality verification,” *Proc. VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, 2018.
- [2] E. Breck, M. Zinkevich, N. Polyzotis, S. Whang, and S. Roy, “Data validation for machine learning,” in *Proc. SysML Conference*, 2019.
- [3] N. Sambasivan, S. Kapania, H. Highfill, D. Akrong, P. K. Paritosh, and L. Aroyo, “‘everyone wants to do the model work, not the data work’: Data cascades in high-stakes AI,” in *Proc. 2021 CHI Conference on Human Factors in Computing Systems*, 2021.
- [4] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu, M. Yasunaga, Y. Zhang, D. Narayanan, Y. Wu, A. Kumar *et al.*, “Holistic evaluation of language models,” *Transactions on Machine Learning Research*, 2023.
- [5] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-Judge with MT-Bench and chatbot arena,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [6] P. W. Koh, S. Sagawa, H. Marklund, S. M. Xie, M. Zhang, A. Balsubramani, W. Hu, M. Yasunaga, R. L. Phillips, I. Gao, T. Lee, E. David, I. Stavness, W. Guo, B. A. Earnshaw, I. S. Haque, S. Beery, J. Leskovec, A. Kundaje, E. Pierson, S. Wang, P. Mostofi, and P. Liang, “WILDS: A benchmark of in-the-wild distribution shifts,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2021.
- [7] D. Hendrycks and T. Dietterich, “Benchmarking neural network robustness to common corruptions and perturbations,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.
- [8] R. Taori, A. Dave, V. Shankar, N. Carlini, B. Recht, and L. Schmidt, “Measuring robustness to natural distribution shifts in image classification,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [9] S. Schelter, T. Rukat, and F. Bießmann, “JENGA: A framework to study the impact of data errors on the predictions of machine learning models,” in *Proc. EDBT*, 2021.
- [10] N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, “Data lifecycle challenges in production machine learning: A survey,” *ACM SIGMOD Record*, vol. 47, no. 2, pp. 17–28, 2018.
- [11] A. Ng *et al.*, “Data-centric AI competition,” 2021, neurIPS Track on Datasets and Benchmarks.
- [12] D. Zha, Z. P. Bhat, K.-H. Lai, F. Yang, and X. Hu, “Data-centric AI: Perspectives and challenges,” in *Proc. SIAM Int. Conf. Data Mining (SDM)*, 2023.
- [13] T. C. Redman, “If your data is bad, your machine learning tools are useless,” *Harvard Business Review*, Apr. 2018.
- [14] S. Hegselmann, A. Buendia, H. Lang, M. Agrawal, X. Jiang, and D. Sontag, “TabLLM: Few-shot classification of tabular data with large language models,” in *Proc. Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2023.
- [15] V. Borisov, K. Sessler, T. Leemann, M. Pawelczyk, and G. Kasneci, “Language models are realistic tabular data generators,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.
- [16] Olist and A. Sionek, “Brazilian E-Commerce public dataset by olist,” Kaggle, 2018, <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>.
- [17] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [18] B. Efron, “Bootstrap methods: Another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [19] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [20] J. Cohen, “A power primer,” *Psychological Bulletin*, vol. 112, no. 1, pp. 155–159, 1992.